

Why use two moving averages?

A single moving average tells us about the general direction of the market, but two moving averages allow us to identify the changes in a trend. For example, if we consider 20 days and 50 days for short-term and long-term moving averages, the 20-day moving average will react relatively quickly to changes in price, while the 50-day moving average will change more slowly. The relationship between them gives us a simple trading rule:

20-day SMA > 50-day SMA → Bullish → Hold

...and:

20-day SMA < 50-day SMA → Bearish → Exit

The actual crossing of the two averages is called a crossover.

Golden Cross and Death Cross

Two popular names for the Moving Average Crossover Strategy are Golden Cross and Death Cross. When the shorter MA crosses above the longer MA the situations are known as Golden Crosses and are usually seen as a positive signal. On the other hand, when the shorter MA crosses below the longer MA the incidents known as Death Crosses and are usually seen as a negative signal.

The periods used in the calculation of the moving average are not always the same and can range from 5/20 and 10/30, to 20/50 and 50/200.

The crossover between the short-term and long-term moving averages helps us to determine the positioning strategy for trading. This has been summarised in Figure 1.

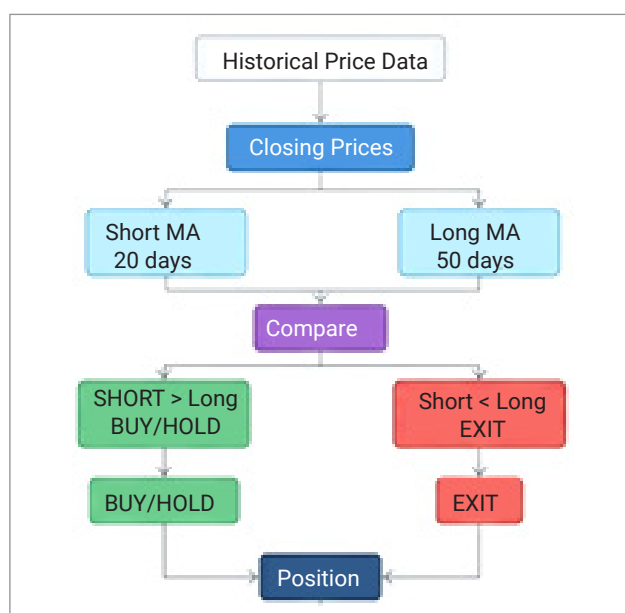


Figure 1: Positioning strategy

This simple workflow forms the basis of our Python implementation.

Obtaining market data with Python

For a study, historical market data can be obtained using *yfinance*.

```
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
ticker = "TCS.NS"
df = yf.download(
    ticker,
    start="2020-01-01",
    end="2025-01-01",
    auto_adjust=True,
    progress=False,
    threads=False
)
print(df.head())
```

Close	High	Low	Open	Volume
TCS.NS	TCS.NS	TCS.NS	TCS.NS	TCS.NS
1831.11	1844.879	1819.621	1831.447	1354908
1822.704	1841.543	1815.566	1841.543	2380752
1859.029	1877.91	1828.069	1828.069	4655761
1858.86	1880.401	1848.258	1862.703	3023209
1863.422	1870.856	1844.795	1858.903	2429317

Before proceeding further, always check that the download was successful:

```
if df.empty:
    raise ValueError("No market data was downloaded.")
```

The resulting DataFrame may have MultiIndex columns depending upon the *yfinance* package version. In case it's needed, get the data for a single ticker/attribute using the *xs()* method.

```
if isinstance(df.columns, pd.MultiIndex):
    df = df.xs(ticker, axis=1, level="Ticker")
```

Then remove all null recordsets:

```
df = df.dropna()
```

Calculating the moving averages

We now calculate the short and long moving averages for two moving window sizes -- 20 and 50: